NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING
MIDTERM ASSESSMENT FOR
Semester 1 AY2023/24

CS1101S Programming Methodology

Time Allowed: 1 hour 30 minutes

INSTRUCTIONS TO CANDIDATES

1. This question paper contains 24 questions in 7 sections, and comprises 16 printed pages, including this page.
2. You are also provided with an answer sheet on which to write your answers.
3. Use a pen or pencil to write your Student Number in the designated space on the front page of the answer sheet, and shade the corresponding circle completely in the grid for each digit or letter. **DO NOT WRITE YOUR NAME.**
4. You must submit only the answer sheet and no other documents. Do not tear off any pages from the answer sheet.
5. All questions must be answered in the space provided in the answer sheet; no extra sheets will be accepted as answers.
6. Write legibly with a pen or pencil; do not use red ink. Untidiness will be penalized.
7. For each multiple-choice question, shade the circle corresponding to the one correct answer completely. The number of marks awarded is shown next to each question. There is no penalty for an incorrect answer.
8. The full score of this assessment is **100 marks**.
9. This is a **closed-book** assessment, but you are allowed to bring one double-sided A4 / letter-sized sheet of handwritten or printed notes.
10. Where programs are required, write them in the Python §2 language. A reference of some of the pre-declared functions is given in the appendix of the question paper.
11. Unless otherwise stated, all programs use the Python §2 language.
12. In any question, unless it is specifically allowed, your answer must not use functions given in, or written by you for, other questions.

Section A: Linked-List and Box Notations [8 marks]

1. [4 marks] What is the result of evaluating the following Python program in box notation?

```
lst = llist(1, 2)
pair(head(lst), pair(tail(lst), tail(tail(lst))))
```

(a.) [1, [2, None]]

(b.) [[[1, 2], None], None]

(c.) [[1, 2, None], None]

(d.) [[1, [2, [None]]], None]

(e.) [1, [[2, None], None]]

(f.) It is impossible to express the result in box notation.

2. [4 marks] What is the result of evaluating the following Python program in linked-list notation?

```
lst = llist(1, 2)

def through(curr):
    return (lst if is_none(curr)
            else pair(tail(curr), through(tail(curr))))

through(lst)
```

(a.) `llist(llist(2), None, llist(1, 2))`

(b.) `llist(llist(2), None, 1, 2)`

(c.) `llist([2, None], llist(1, 2))`

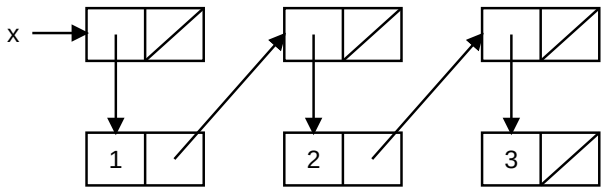
(d.) `llist(1, None, 1, 2)`

(e.) It is impossible to express the result in linked-list notation.

(f.) The program leads to an error.

Section B: Box and Pointer Diagrams [12 marks]

3. [4 marks] Which of the following programs produces the pairs shown in the given box-and-pointer diagram?



(a.) `x = llist(llist(1, llist(2, llist(3))))`

(b.) `x = pair(pair(1, pair(2, pair(3, None))), None)`

(c.) `x = llist(llist(1, llist(2, 3)))`

(d.) `x = llist(llist(1, 2, llist(3)))`

(e.) `x = llist(1, llist(2, llist(3)))`

(f.) `x = llist(llist(1, llist(2, 3), None))`

4. [4 marks] Draw the box-and-pointer diagram for the value of `x` after the evaluation of the following program. Clearly show where `x` is pointing to.

```
x = pair(llist(pair(1, 1)), llist(1))
```

5. [4 marks] Draw the box-and-pointer diagram for the value of `x` after the evaluation of the following program. Clearly show where `x` is pointing to.

```
lst = llist(llist(pair(None, None)))
x = reduce(lambda x, y: llist(y, x), 0, lst)
```

Section C: Data Structures - Dictionaries [9 marks]

A dictionary data structure provides a way to store data entries where each entry consists of a key and a value. We represent a key-value entry as a pair in Python, where the head of the pair is the key and the tail is the value. A dictionary is represented as a linked list of pairs in Python.

```
student_names = llist(pair("A101X", "Alice"),
                      pair("A105Y", "Bob"),
                      pair("A102Z", "Cat"))

student_marks = llist(pair("A101X", 78),
                      pair("A105Y", 69),
                      pair("A102Z", 84))
```

6. [5 marks] Complete the declaration of function `is_entry`, which takes in two arguments: a key and a dictionary, and returns `True` if the dictionary has an entry corresponding to the input key, or returns `False` otherwise.

For example, `is_entry("A105Y", student_names)` returns `True`, and `is_entry("A109W", student_names)` returns `False`.

Write each numbered blank in the matching space on the answer sheet.

```
def is_entry(key, dictionary):
    return reduce(
        ___[1]___,
        ___[2]___,
        dictionary)
```

7. [4 marks] Complete the declaration of function `remove_entry`, which takes in two arguments: a key and a dictionary, and returns a dictionary with the entry corresponding to the input key removed, if such an entry exists in the dictionary. You may assume that keys are unique in the dictionary.

For example, `remove_entry("A105Y", student_names)` returns `llist(pair("A101X", "Alice"), pair("A102Z", "Cat"))`.

Write each numbered blank in the matching space on the answer sheet.

```
def remove_entry(key, dictionary):
    return filter(
        ___[1]___,
        dictionary)
```

Section D: Orders of Growth [16 marks]

For each question in this section, choose the order of growth that best characterizes the running time of the function. Note that `N` is a positive integer value.

8. [4 marks]

```
def fun(N):
    return (llist(1) if N <= 1
            else enum_llist(1, N) if N % 2 == 0
            else map(lambda x: enum_llist(1, N), enum_llist(1, N)))
```

(a.) $\Omega(N^2)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N^2)$

(e.) $\Theta(N^3)$

(f.) $O(N)$

(g.) $O(N \log N)$

(h.) $O(N^2)$

9. [3 marks]

```
def fun(N):
    return 1 if N <= 1 else N * fun(N / 4)
```

(a.) $\Theta(1)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N \log N)$

(e.) $\Theta(N^2)$

(f.) $\Theta(N^2 \log N)$

(g.) $\Theta(N^3)$

(h.) $\Theta(k^N)$, where k is some constant greater than `1`

10. [3 marks]

```
def fun(N):
    return 1 if N <= 1 else fun(N - 10) + fun(N - 10)
```

(a.) $\Theta(1)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N \log N)$

(e.) $\Theta(N^2)$

(f.) $\Theta(N^2 \log N)$

(g.) $\Theta(N^3)$

(h.) $\Theta(k^N)$, where k is some constant greater than 1

11. [3 marks]

```
def fun(N):
    if N <= 1:
        return 1
    else:
        L = enum_llist(1, N)
        return 3 * fun(N - 10)
```

(a.) $\Theta(1)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N \log N)$

(e.) $\Theta(N^2)$

(f.) $\Theta(N^2 \log N)$

(g.) $\Theta(N^3)$

(h.) $\Theta(k^N)$, where k is some constant greater than 1

12. [3 marks]

```
def fun(N):
    if N <= 1:
        return 1
    else:
        L = enum_llist(1, math_floor(N))
        return N * fun(N / 2)
```

(a.) $\Theta(1)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N \log N)$

(e.) $\Theta(N^2)$

(f.) $\Theta(N^2 \log N)$

(g.) $\Theta(N^3)$

(h.) $\Theta(k^N)$, where k is some constant greater than 1

Section E: Linked-List Shifting [13 marks]

13. [4 marks] Complete the declaration of function `circular_left_shift`, which takes as argument a linked list, and returns a linked list that is the result of circularly shifting the input linked list to the left by one position.

For example, `circular_left_shift(llist(3, 4, 5, 6, 7))` returns `llist(4, 5, 6, 7, 3)`. A one-element linked list and `None` are returned unchanged. You must not use `append` or declare another function. Your function must run in $O(n)$ time, where n is the length of the input.

Write each numbered blank in the matching space on the answer sheet.

```
def circular_left_shift(xs):
    if is_none(xs):
        return xs
    else:
        ___[1]___
```

14. [5 marks] Complete the declaration of function `circular_right_shift`, which takes as argument a linked list, and returns a linked list that is the result of circularly shifting the input linked list to the right by one position.

For example, `circular_right_shift(llist(3, 4, 5, 6, 7))` returns `llist(7, 3, 4, 5, 6)`. A one-element linked list and `None` are returned unchanged. You must not use `append` or declare another function. Your function must run in $O(n)$ time, where n is the length of the input.

Write each numbered blank in the matching space on the answer sheet.

```
def circular_right_shift(xs):
    if is_none(xs):
        return xs
    else:
        ___[1]___
```

15. [4 marks] Which of the following is a correct way to reverse a linked list `xs`?

(a.) `reduce(lambda x, ys: circular_right_shift(pair(x, ys)), None, xs)`

(b.) `reduce(lambda x, ys: circular_left_shift(pair(x, ys)), None, xs)`

(c.) `reduce(lambda x, ys: pair(head(ys), circular_right_shift(pair(x, tail(ys)))), None, xs)`

(d.) `reduce(lambda x, ys: pair(head(ys), circular_left_shift(pair(x, tail(ys)))), None, xs)`

(e.) `reduce(lambda x, ys: pair(x, circular_right_shift(ys)), None, xs)`

(f.) `reduce(lambda x, ys: pair(x, circular_left_shift(ys)), None, xs)`

Section F: Trees [24 marks]

16. [3 marks] Consider the following constant declarations for data structures.

```
data1 = llist(1, pair(2, None), llist(4, 3))
data2 = llist(pair(1, 2), 3, llist(4))
data3 = llist(1, llist(2, llist(3, llist(4))))
```

Which of the above data structures is or are trees?

(a.) data1 only

(b.) data1 and data2 only

(c.) data1 and data3 only

(d.) data2 and data3 only

(e.) data1, data2, and data3

(f.) None of the data structures

17. [5 marks] Consider the function `new_tree_sum`.

```
def new_tree_sum(tree):
    return (print_value(0) if is_none(tree)
            else new_tree_sum(tail(tree))
                 + (new_tree_sum(head(tree))
                    if is_llist(head(tree))
                    else print_value(head(tree))))

def print_value(x):
    print(x)
    return x

tree = llist(1, llist(2, llist(3, 4), 5))
new_tree_sum(tree)
```

What is the sequence of values printed by the `print` function?

(a.) `0 0 0 5 4 3 2 1`

(b.) `0 0 5 0 4 3 2 1`

(c.) `0 5 0 0 4 3 2 1`

(d.) `1 2 3 4 5 0 0 0`

(e.) `1 2 3 4 0 5 0 0`

(f.) `1 2 3 4 0 0 5 0`

(g.) `5 4 3 2 1 0 0 0`

(h.) `5 0 4 3 0 2 0 1`

18. [5 marks] Complete the declaration of function `max_tree`, which takes as argument a tree of positive numbers, and returns the maximum value in that tree. It returns `0` for the empty tree. You must not use higher-order linked-list processing functions; you may find the pre-declared function `max` useful.

For example, `max_tree(llist(1, llist(2, llist(3, 4), 5)))` returns `5`, and `max_tree(llist(5, llist(6, llist(3, 2), 1)))` returns `6`.

Write each numbered blank in the matching space on the answer sheet.

```
def max_tree(tree):
    return (___[1]___ if is_none(tree)
            else ___[2]___ if ___[3]___
            else ___[4]___)
```

19. [5 marks] Recall our implementation of `accumulate_tree` in class.

```
def accumulate_tree(f, op, initial, tree):
    return reduce(
        lambda x, ys:
        (op(f(x), ys) if not is_llist(x)
         else op(accumulate_tree(f, op, initial, x), ys)),
        initial,
        tree)

def print_value(x):
    print(x)
    return x

tree4 = llist(2, llist(1, 3, llist(5, 4), llist(6)))
accumulate_tree(print_value, lambda x, y: x + y, 0, tree4)
```

What is the sequence of values printed by the `print` function?

(a.) `1 2 3 4 5 6`

(b.) `2 1 3 5 4 6`

(c.) `2 1 3 6 5 4`

(d.) `6 5 4 2 1 3`

(e.) `6 5 4 1 3 2`

(f.) `6 4 5 3 1 2`

20. [6 marks] Complete the declaration of the function `replace_elem`, which takes in three arguments: an element to be replaced `old_e`, a new element `new_e`, and a tree, and returns a tree with all elements `old_e` replaced with `new_e` using the `accumulate_tree` function.

For example, `replace_elem(1, 5, llist(1, llist(4, llist(3, 2), 1)))` returns `llist(5, llist(4, llist(3, 2), 5))`.

Write each numbered blank in the matching space on the answer sheet.

```
def replace_elem(old_e, new_e, tree):
    return accumulate_tree(
        ___[1]___,
        ___[2]___,
        ___[3]___,
        ___[4]___)
```

Section G: Functions [18 marks]

21. [4 marks] Consider the following function `mystery`.

```
def mystery(x, y):
    return lambda z: z >= x and z <= y
```

Which one of the following expressions returns `True`?

(a.) `mystery(5, 6)(10)`

(b.) `mystery(10, 5)(6)`

(c.) `mystery(6, 5)(10)`

(d.) `mystery(5, 10)(6)`

(e.) `mystery(6, 10)(5)`

(f.) `mystery(10, 6)(5)`

22. [5 marks] Consider the following function `enigma`.

```
def enigma(a):
    return lambda b: lambda c: c(a(b))
```

Which one of the following expressions returns `17`?

(a.) `enigma(lambda x: x * x)(4)(lambda y: y + 1)`

(b.) `enigma(lambda x: x + 1)(4)(lambda y: y * y)`

(c.) `enigma(4)(lambda x: x + 1)(lambda y: y * y)`

(d.) `enigma(4)(lambda x: x * x)(lambda y: y + 1)`

(e.) `enigma(lambda x: x + 1)(lambda y: y * y)(4)`

(f.) `enigma(lambda x: x * x)(lambda y: y + 1)(4)`

23. [5 marks] The function `fun_filter` takes as arguments a linked list of predicates `ps` and a value `x`, and returns the linked list of predicates in `ps` for which the predicates hold on `x`.

For example, with

```
my_fs = llist(lambda y: y == 0, lambda y: y < 10,
              lambda y: y > 20, lambda y: y % 2 == 0)
fun_filter(my_fs, 6)
# returns the predicates lambda y: y < 10 and lambda y: y % 2 == 0
```

Which one of the following is a correct implementation of `fun_filter`?

(a.) `fun_filter = lambda ps, x: filter(x, ps)`

(b.) `fun_filter = lambda ps, x: filter(ps, x)`

(c.) `fun_filter = lambda ps, x: filter(lambda f: (lambda x: x if f(x) else f), ps)`

(d.) `fun_filter = lambda ps, x: filter(lambda f: f if f(x) else x, ps)`

(e.) `fun_filter = lambda ps, x: filter(lambda f: f(x), ps)`

(f.) `fun_filter = lambda ps, x: filter(lambda f: lambda x: f(x), ps)`

24. [4 marks] Alonzo Church represented numbers using functions. The Church numeral `three` applies a function `f` three times to a value `x`:

```
zero = lambda f: lambda x: x
three = lambda f: lambda x: f(f(f(x)))
succ = lambda n: lambda f: lambda x: f(n(f)(x))
# succ(three) is equivalent to lambda f: lambda x: f(f(f(f(x))))
```

Moses Schönfinkel and Haskell Curry showed that any computable function can be represented using the following functions:

```
S = lambda x: lambda y: lambda z: x(z)(y(z))
K = lambda x: lambda y: x
I = lambda x: x
```

Any computable function can be represented by combining these functions using function application. In fact, `I` is not needed: an SKI combination `J` that does not use `I` can behave exactly like it.

For example, the successor function can be represented as `succ = S(S(K(S))(K))`, without using `I`. Likewise, `J(three)` should return `three`.

Which declaration makes `J(A)` return `A` for every argument `A`?

(a.) `J = S(S)(S)`

(b.) `J = K(S)(K)`

(c.) `J = S(K)(K)`

(d.) `J = K(S)(S)`

(e.) `J = S(S)(K)`

(f.) `J = K(K)(K)`

(g.) `J = K(K)(S)`

APPENDIX

Python §2 Function Reference

Pairs, linked lists, higher-order functions, type predicates, and mathematical helpers

---

The following functions are pre-declared in Python §2. `None` denotes the empty linked list; linked-list positions start at zero. The linked-list processing functions below use iterative processes, with the stated time and space bounds. Bounds assume constant-time element operations.

Pairs and linked lists

`pair(x, y)` Constructs a pair with head `x` and tail `y`. Time and space $O(1)$.

`is_pair(x)` Returns `True` if `x` is a pair, and `False` otherwise. Time and space $O(1)$.

`head(x)` Returns the first component of pair `x`. Time and space $O(1)$.

`tail(x)` Returns the second component of pair `x`. Time and space $O(1)$.

`is_none(xs)` Returns `True` if `xs` is `None`, and `False` otherwise. Time and space $O(1)$.

`is_llist(x)` Returns `True` if `x` is a linked list as defined in lectures, and `False` otherwise. Time $O(n)$ and space $O(1)$, where `n` is the length of the chain of tail operations that can be applied to `x`.

`llist(x1, …, xn)` Constructs a linked list of `n` elements, in the given order. Time and space $O(n)$; the result contains `n` pairs.

`length(xs)` Returns the length of linked list `xs`. Time $O(n)$ and space $O(1)$, where `n` is its length.

Search and selection

`member(x, xs)` Returns the first suffix whose head is identical to `x`, or `None` if none exists. Scalar values match by value; pairs and functions match by object identity. Time $O(n)$ and space $O(1)$.

`remove(x, xs)` Returns a linked list with the first matching element removed, if present. Matching uses scalar value equality and pair/function identity. Time and space $O(n)$.

`remove_all(x, xs)` Returns a linked list with every matching element removed, using scalar value equality and pair/function identity. Time and space $O(n)$.

`filter(pred, xs)` Returns the elements of `xs` for which unary predicate `pred` returns `True`, preserving their order. Time and space $O(n)$.

`enum_llist(start, end)` Enumerates numbers from `start` in steps of 1 until the next number exceeds `end`. The end is inclusive: `enum_llist(2, 5)` returns `llist(2, 3, 4, 5)`. Time and space $O(n)$ in the result length.

`llist_ref(xs, n)` Returns the element at position `n`, counting from zero. Time $O(n)$ and space $O(1)$.

Building and processing linked lists

`map(f, xs)` Returns the result of applying unary function `f` to every element of `xs`, in order. Time and space $O(n)$.

`build_llist(f, n)` Constructs `n` elements by applying `f` to the numbers 0 through `n - 1`, in order. Time and space $O(n)$.

`for_each(f, xs)` Applies `f` to each element of `xs` in order, then returns `True`. Time $O(n)$ and space $O(1)$.

`reverse(xs)` Returns the elements of `xs` in reverse order. Time and space $O(n)$; the process is iterative, but the result itself occupies $O(n)$ space.

`append(xs, ys)` Returns the elements of `xs` followed by those of `ys`. Time and space $O(n)$ in the length of `xs`. A definition supplied in a question overrides this pre-declared implementation.

`reduce(op, initial, xs)` Folds from right to left. First `op` is applied to the last element and `initial`, then to the preceding element and that result, and so on. For example:

```
reduce(op, initial, llist(1, 2, 3))
# equals op(1, op(2, op(3, initial)))
```

Time and space $O(n)$, assuming `op` takes constant time. `reduce` here is the pre-declared linked-list right fold.

Types, output, and mathematics

`is_boolean(x), is_number(x), is_string(x), is_function(x)` Return `True` exactly when `x` has the named type, and `False` otherwise. `is_integer`, `is_float`, and `is_complex` test the corresponding numeric types.

`print(*objects)` Prints the objects followed by a newline and returns `None`. Where a question needs to print and return a value, it defines `print_value` explicitly.

`str(object), repr(object)` Return informal and debugging string representations, respectively.

`llist_to_string(xs)` Returns a string representing `xs` in box notation.

`math_floor(x)` Returns the largest integer less than or equal to `x`.

`math_sqrt(x)` Returns the square root of `x`.

`math_pow(base, exp)` Returns `base` raised to the power `exp`.

`min(x1, …, xn), max(x1, …, xn)` Return the smallest and largest supplied values, respectively. Time $O(n)$ and space $O(1)$.

END OF PAPER